

TP8 — Implémenter un système sécurisé de gestion des secrets

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description



.vault_pass = la cle qui ouvre le coffre
vault.yml = le coffre, contient le vrai secret (le token API)
Le token ne passe jamais par .vault_pass — il vit uniquement dans vault.yml

Figure 1: Flux complet Ansible Vault

Ce TP protège un secret (un token d'API) avec Ansible Vault plutôt que de le laisser en clair dans les fichiers versionnés. On ajoute une authentification par token au mock d'API (TP4), on chiffre ce token, et on configure le rôle base_config pour l'envoyer à chaque requête — la même logique que pour un vrai credential DNA Center/ACI/SD-WAN plus tard dans la formation.

Prérequis

- TP1 à TP7 validés

Étapes

Étape 1 — Créer sa branche pour ce TP

```
cd netops-project-<nom-groupe>
git checkout dev
```

```
git pull origin dev
git checkout -b <prenom>-<nom>-jour2-tp8
```

Étape 2 — Ajouter une authentification par token au mock d'API

```
cat > scripts/mock_api.py << 'EOF'
import os
import sys
from flask import Flask, jsonify, request

app = Flask(__name__)

SITE_NAME = os.environ.get("SITE_NAME", "site-inconnu")
API_TOKEN = os.environ.get("API_TOKEN", "changeme")

INTERFACES = {
    "GigabitEthernet0/1": {"status": "up", "ip": "10.0.1.1", "vlan": 10},
    "GigabitEthernet0/2": {"status": "down", "ip": None, "vlan": 20},
}

def check_token():
    return request.headers.get("X-API-Token") == API_TOKEN

@app.route("/health")
def health():
    return jsonify({"site": SITE_NAME, "status": "ok"})

@app.route("/api/devices/<device_id>/interfaces")
def get_interfaces(device_id):
    if not check_token():
        return jsonify({"error": "Token invalide ou manquant"}), 401
    return jsonify({"site": SITE_NAME, "device_id": device_id, "interfaces": INTERFACES})

if __name__ == "__main__":
    port = int(sys.argv[1]) if len(sys.argv) > 1 else 5000
    app.run(host="0.0.0.0", port=port, debug=False)
EOF
```

Étape 3 — Isoler le token du lab dans un fichier .env non commité

Le mock d'API simule un équipement réseau ; son propre credential ne doit pas plus se retrouver en clair dans Git que celui d'Ansible. On le sort donc de docker-compose.yml :

```

cat > lab/.env << 'EOF'
API_TOKEN=s3cr3t-token-nexacorp
EOF
echo "lab/.env" >> .gitignore
cat > lab/docker-compose.yml << 'EOF'
services:
  siege-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: siege-router
    environment:
      - SITE_NAME=siege
      - API_TOKEN=${API_TOKEN}
    ports:
      - "5001:5000"
    networks:
      - nexacorp-lab

  agencel-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: agencel-router
    environment:
      - SITE_NAME=agencel
      - API_TOKEN=${API_TOKEN}
    ports:
      - "5002:5000"
    networks:
      - nexacorp-lab

  agence2-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: agence2-router
    environment:
      - SITE_NAME=agence2
      - API_TOKEN=${API_TOKEN}
    ports:
      - "5003:5000"
    networks:
      - nexacorp-lab

networks:
  nexacorp-lab:

```

```
driver: bridge
```

E0F

Docker Compose lit automatiquement `lab/.env` et remplace `${API_TOKEN}` par sa valeur. Résultat : le seul endroit où ce token existe en clair dans le projet est un fichier local jamais commité — le dépôt Git, lui, ne contient que la version chiffrée (Étape 6).

Étape 4 — Redémarrer le lab avec ce changement

```
cd lab
docker compose down
docker compose up -d --build
cd ..
```

Étape 5 — Créer le fichier de mot de passe Vault (jamais commité)

```
echo "<votre-mot-de-passe-vault>" > .vault_pass
chmod 600 .vault_pass
chmod -x .vault_pass
echo ".vault_pass" >> .gitignore
```

Sous WSL, sur un disque Windows monté (`/mnt/c/...`) : tous les fichiers y apparaissent parfois comme exécutables par défaut, ce qui fait qu'Ansible essaie de lancer `.vault_pass` comme un script plutôt que de lire son contenu (`Exec format error`). `chmod -x .vault_pass` corrige ça. Si le montage réapplique systématiquement le bit exécutable, éditez `/etc/wsl.conf` (dans le terminal Ubuntu) :

```
[automount]
options = "metadata"
```

Puis, depuis PowerShell : `wsl --shutdown`, et rouvrez le terminal Ubuntu.

Si vous relancez WSL (`wsl --shutdown` ou redémarrage), le dossier `/tmp` est réinitialisé — un fichier temporaire créé avant (comme `/tmp/vault_plain.yml` à l'étape suivante) disparaît et doit être recréé.

Étape 6 — Restructurer `group_vars/all.yml` en dossier, et chiffrer le secret

Ansible ne charge automatiquement que `group_vars/<nom-de-groupe>.yml` **ou** un dossier `group_vars/<nom-de-groupe>/` — pas un nom de fichier arbitraire comme `vault.yml`. On restructure donc `group_vars/all.yml` :

```
mkdir -p inventory/group_vars/all
git mv inventory/group_vars/all.yml inventory/group_vars/all/vars.yml

echo "vault_api_token: s3cr3t-token-nexacorp" > /tmp/vault_plain.yml
ansible-vault encrypt --vault-password-file .vault_pass \
```

```
--output inventory/group_vars/all/vault.yml /tmp/vault_plain.yml
rm /tmp/vault_plain.yml
```

La valeur s3cr3t-token-nexacorp doit être **exactement identique** à celle mise dans lab/.env à l'étape 3.

Vérifiez que le fichier est bien chiffré :

```
cat inventory/group_vars/all/vault.yml
```

Vous devez voir un bloc \$ANSIBLE_VAULT;1.1;AES256, pas la valeur en clair.

Étape 7 — Vérifier qu'on peut le déchiffrer avec le bon mot de passe

```
ansible-vault view --vault-password-file .vault_pass inventory/group_vars/all/vault.yml
```

Doit afficher vault_api_token: s3cr3t-token-nexacorp en clair.

Étape 8 — Mettre à jour le rôle pour envoyer le token

```
cat > roles/base_config/tasks/main.yml << 'EOF'
---
- name: Verifier que le site repond (health check)
  ansible.builtin.uri:
    url: "{{ api_url }}/health"
    method: GET
    return_content: yes
    register: health_result

- name: Afficher le statut du site
  ansible.builtin.debug:
    msg: "Site {{ health_result.json.site }} : {{ health_result.json.status }}"

- name: Recuperer l'etat des interfaces du site (authentifie)
  ansible.builtin.uri:
    url: "{{ api_url }}/api/devices/{{ device_id_prefix }}-{{ inventory_hostname }}-01/interfaces"
    method: GET
    headers:
      X-API-Token: "{{ vault_api_token }}"
    return_content: yes
    register: interfaces_result

- name: Identifier les interfaces en panne
  ansible.builtin.set_fact:
    interfaces_down: "{{ interfaces_result.json.interfaces | dict2items
      | selectattr('value.status', 'equalto', 'down') | map(attribute='key') | list }}"

- name: Alerter si des interfaces sont en panne
  ansible.builtin.debug:
    msg: "ATTENTION sur {{ inventory_hostname }} : interfaces en panne -> {{ interfaces_down }}"
```

```
when: interfaces_down | length > 0
EOF
```

Étape 9 — Tester SANS le mot de passe Vault (doit échouer)

```
ansible-playbook playbooks/site.yml
```

Résultat attendu : [ERROR]: Attempting to decrypt but no vault secrets found. — sans le mot de passe, Ansible refuse même d'essayer de lire la variable chiffrée.

Si à la place vous obtenez un warning Ansible is being run in a world writable directory ... ignoring it as an ansible.cfg source, c'est que la variable ANSIBLE_CONFIG mise en place au TP7 n'est plus active dans ce terminal. Vérifiez avec `echo $ANSIBLE_CONFIG`, et si c'est vide :

```
export ANSIBLE_CONFIG=/mnt/c/Users/<votre-nom>/Desktop/netops-project/ansible.cfg
echo 'export ANSIBLE_CONFIG=/mnt/c/Users/<votre-nom>/Desktop/netops-project/ansible.cfg' >> ~/.bashrc
source ~/.bashrc
```

Étape 10 — Tester AVEC le mot de passe Vault (doit réussir)

```
ansible-playbook --vault-password-file .vault_pass playbooks/site.yml
```

Résultat attendu : les 3 sites (siege, agence1, agence2) passent en ok, avec le message d'alerte sur l'interface en panne pour chacun.

Étape 11 — Committer et pousser

```
git add scripts/mock_api.py roles/base_config/tasks/main.yml inventory/group_vars/ lab/docker-compose.yml .gitignore
git commit -m "TP8 : authentication par token protegee avec Ansible Vault"
git push -u origin <prenom>-<nom>-jour2-tp8
```

Ouvrir une pull request <prenom>-<nom>-jour2-tp8 → dev.

Point d'attention pour l'équipe : tous les étudiants doivent utiliser le même mot de passe Vault (convention partagée), et la même valeur de token dans `lab/.env` (local, non commité) et dans `vault.yml` (chiffré, commité) — sinon le déploiement d'un camarade échouera après un merge.

Comprendre le TP

Deux fichiers ont des rôles complètement différents, à ne pas confondre :

- **.vault_pass** contient le mot de passe choisi par vous — la clé qui ouvre le coffre-fort. Il ne contient jamais le secret lui-même.
- **inventory/group_vars/all/vault.yml** contient le vrai secret (le token API), chiffré avec AES256 en utilisant le mot de passe de `.vault_pass`. C'est ce fichier qu'on committe dans Git, en toute sécurité, puisqu'il est illisible sans la clé.

À chaque exécution du playbook avec `--vault-password-file .vault_pass` : Ansible lit le mot de passe, l'utilise pour déchiffrer `vault.yml` en mémoire (jamais réécrit en clair sur le disque), ce qui rend la variable `vault_api_token` disponible comme n'importe quelle autre variable. Le rôle l'utilise alors dans l'en-tête HTTP `X-Api-Token` de sa requête vers l'API du site, qui compare cette valeur à son propre `API_TOKEN` (défini via `lab/.env`) pour autoriser ou refuser l'accès.

Le Vault protège une copie précise, dans un endroit précis : ici, le dépôt Git partagé de l'équipe d'automatisation. En conditions réelles, la valeur d'origine viendrait d'un coffre-fort d'entreprise (Azure Key Vault, 1Password, HashiCorp Vault) ; un humain la récupère une fois pour la chiffrer dans `vault.yml`, qu'Ansible peut ensuite utiliser à volonté sans jamais exposer le secret en clair dans l'historique du dépôt.

Validation

- ☐ `lab/.env` (avec le token) existe en local mais est ignoré par Git
- ☐ `inventory/group_vars/all/vault.yml` est chiffré (`$ANSIBLE_VAULT;1.1;AES256`), jamais en clair
- ☐ `.vault_pass` existe en local (droits 600, sans bit exécutable) mais est ignoré par Git
- ☐ `ansible-vault view --vault-password-file .vault_pass ...` affiche bien le secret en clair
- ☐ Le playbook échoue clairement sans `--vault-password-file` (`Attempting to decrypt but no vault secrets found`)
- ☐ Le playbook réussit avec `--vault-password-file .vault_pass`
- ☐ Une pull request `<prenom>-<nom>-jour2-tp8` → dev est ouverte